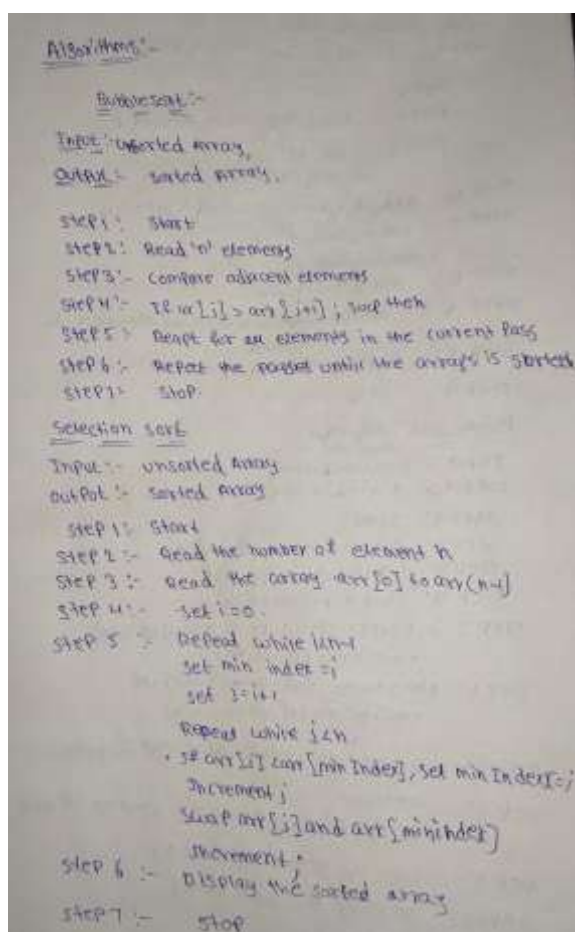


practical: 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
19/08/2026	

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:



Insertion sort - algorithm

Input :- unsorted array
output :- sorted array

STEP 1:- START
STEP 2:- Read the array element A
STEP 3:- set $i=1$
 set $key=A[i]$ and $j=i-1$
STEP 4:- Repeat while $j>0$ and $A[j]>key$
STEP 5:- set $A[j+1]=A[j]$
 Decrement j
STEP 6:- set $A[j+1]=key$
STEP 7:- Repeat steps 4-6 until all elements are sorted
STEP 8:- Display the sorted array
STEP 9:- STOP

Mergesort - algorithm

Input :- unsorted array
output :- sorted array

STEP 1:- START
STEP 2:- Read the array elements A
STEP 3:- set $low=0$, $high=n-1$
STEP 4:- find $mid=(low+high)/2$
STEP 5:- Apply merge sort to the right half
 • mergesort(A, low, mid)
STEP 6:- Apply merge sort to the right half
 • mergesort(A, mid+1, high)
STEP 7:- merge both sorted halves using i, j and k
STEP 8:- compare $A[i]$, $A[j]$ and copy the smaller element
STEP 9:- copy the remaining elements
STEP 10:- Display the sort array
STEP 11:- STOP

Quick sort algorithm

Input :- unsorted array
output :- sorted array

STEP 1:- START
STEP 2:- Read the array elements 'A'
STEP 3:- set $low=0$, $high=n-1$
STEP 4:- Search an element as pivot
STEP 5:- set $i=low$, $j=high$
STEP 6:- Repeat while $i<j$
 • If $A[i]< pivot$, increment
 • If $A[j]> pivot$, decrement
 • If $i < j$ swap $A[i]$ and $A[j]$
 then increment i and decrement j
STEP 7:- Apply quicksort to the left half
 quicksort(A, low, i-1)
STEP 8:- right half quicksort(A, i, high)
STEP 9:- Display the sorted array
STEP 10:- STOP

Program:

```
#include <iostream>

using namespace std;

// ----- BUBBLE SORT -----

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
        }
    }
}

// ----- SELECTION SORT -----

void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
    }
}
```

```
}

swap(arr[i], arr[minIndex]);
}
}

// ----- INSERTION SORT -----
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

// ----- MERGE SORT -----
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];
    int i = low;
```

```
int j = mid + 1;
```

```
int k = low;
```

```
while (i <= mid && j <= high)
```

```
{
```

```
    if (arr[i] < arr[j])
```

```
        temp[k++] = arr[i++];
```

```
    else
```

```
        temp[k++] = arr[j++];
```

```
}
```

```
while (i <= mid)
```

```
    temp[k++] = arr[i++];
```

```
while (j <= high)
```

```
    temp[k++] = arr[j++];
```

```
for (i = low; i <= high; i++)
```

```
    arr[i] = temp[i];
```

```
}
```

```
void mergeSort(int arr[], int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```

```
        int mid = (low + high) / 2;
```

```
        mergeSort(arr, low, mid);
```

```
        mergeSort(arr, mid + 1, high);
```

```
        merge(arr, low, mid, high);
    }
}

// ----- QUICK SORT -----

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
```

```
int p = partition(arr, low, high);

quickSort(arr, low, p - 1);
quickSort(arr, p + 1, high);
}
}

// ----- DISPLAY -----
void display(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;
}

// ----- MAIN -----
int main()
{
    int n;
    int original[100];

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> original[i];
```

```
int bubble[100], selection[100], insertion[100];
```

```
int mergeArr[100], quick[100];
```

```
// Copy original array
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    bubble[i] = original[i];
```

```
    selection[i] = original[i];
```

```
    insertion[i] = original[i];
```

```
    mergeArr[i] = original[i];
```

```
    quick[i] = original[i];
```

```
}
```

```
// Sorting
```

```
bubbleSort(bubble, n);
```

```
selectionSort(selection, n);
```

```
insertionSort(insertion, n);
```

```
mergeSort(mergeArr, 0, n - 1);
```

```
quickSort(quick, 0, n - 1);
```

```
// Display results
```

```
cout << "\nBubble Sort:  ";
```

```
display(bubble, n);
```

```
cout << "Selection Sort: ";
```

```
display(selection, n);
```

```
cout << "Insertion Sort: ";
```

```
display(insertion, n);
```



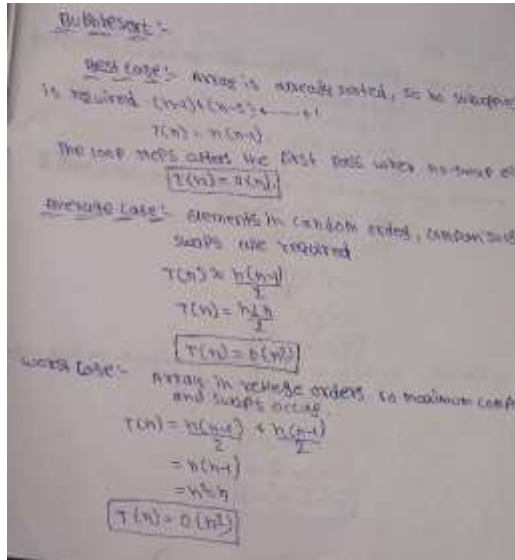
```
cout << "Merge Sort:  ";  
display(mergeArr, n);  
  
cout << "Quick Sort:  ";  
display(quick, n);  
  
return 0;  
}
```

Output:

```
Enter number of elements: 6  
Enter elements: 64 25 12 22 11 90  
  
Bubble Sort:    11 12 22 25 64 90  
Selection Sort: 11 12 22 25 64 90  
Insertion Sort: 11 12 22 25 64 90  
Merge Sort:     11 12 22 25 64 90  
Quick Sort:     11 12 22 25 64 90  
  
=== Code Execution Successful ===
```

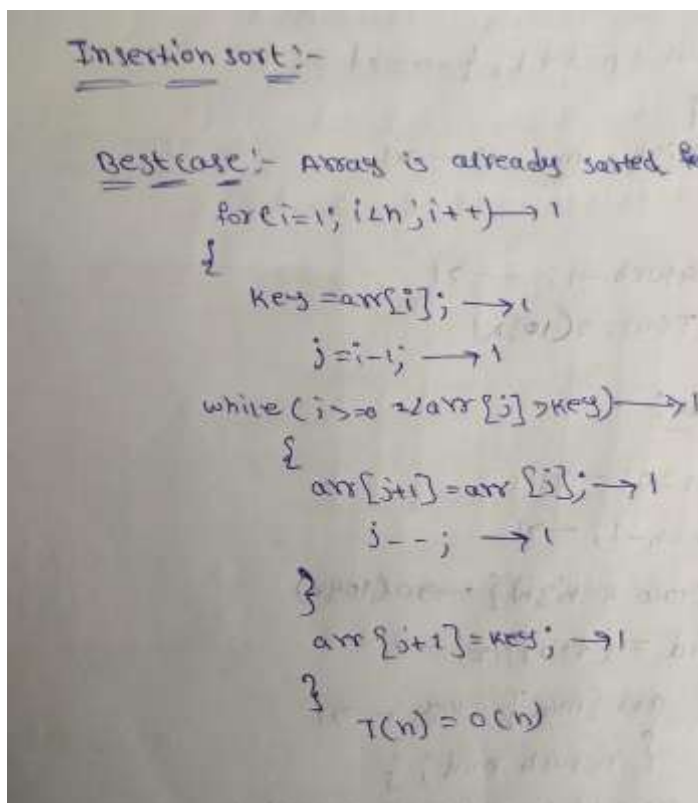
Time Complexity:

BUBBLE SORT:

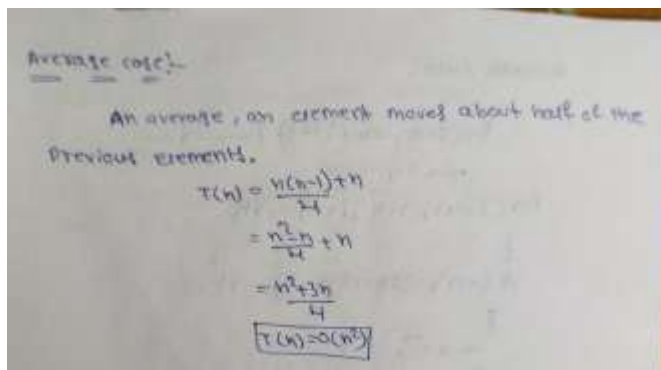


INSERTION SORT:

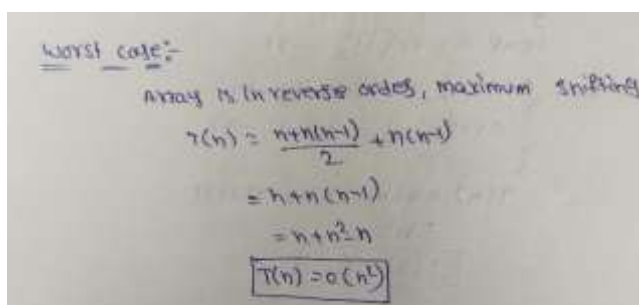
Best Case:



Average Case:

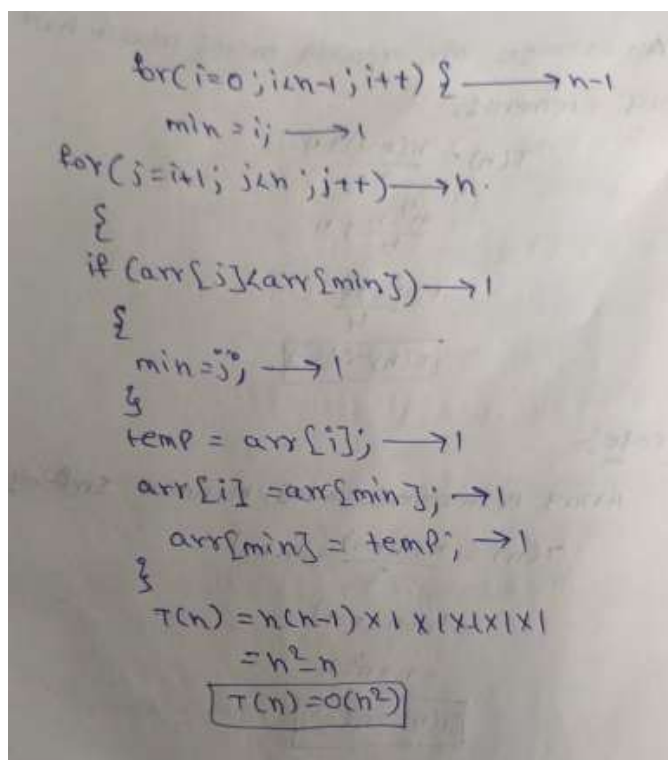


Worst Case:



SELECTION SORT:

Best Case:



Average Case:

```

for(i=0; i<n-1; i++) { → n-1
    min = i; → 1
    for(j=i+1; j<n; j++) → n
    {
        if(arr[j] < arr[min]) → 1
        {
            min = j; → 1
        }
        temp = arr[i]; → 1
        arr[i] = arr[min]; → 1
        arr[min] = temp; → 1
    }
    T(n) = n(n-1) × 1 × 1 × 1 × 1 × 1
        = n² - n
        T(n) = O(n²)
  
```

Worst Case:

```

for(i=0; i<n-1; i++) { → n-1
    min = i; → 1
    for(j=i+1; j<n; j++) → n
    {
        if(arr[j] < arr[min]) → 1
        {
            min = j; → 1
        }
        temp = arr[i]; → 1
        arr[i] = arr[min]; → 1
        arr[min] = temp; → 1
    }
    T(n) = n(n-1) × 1 × 1 × 1 × 1 × 1
        = n² - n
        T(n) = O(n²)
  
```

MERGE SORT:

Best Case, Average Case, Worst Case:

```

merge sort :-
void merge_sort (int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid+1, k = low;
    while (i < mid+1 & j < high) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i < mid+1) {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j < high) {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (i = low; i < high; i++)
        arr[i] = temp[i];
}

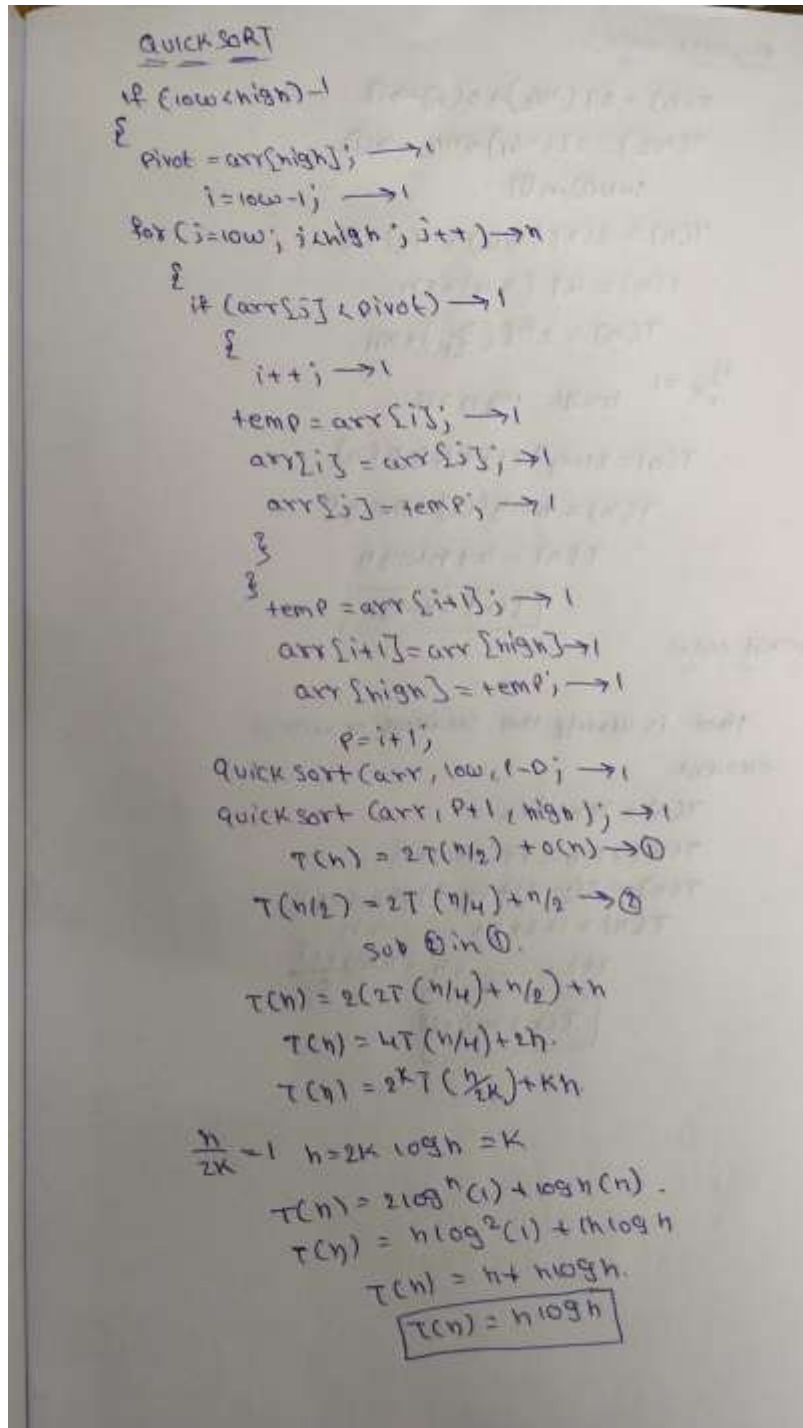
void merge_logic (int low, int high)
{
    if (low < high)
        mid = (low+high)/2;
    merge_logic (low, mid);
    merge_logic (mid+1, high);
    merge (low, mid, high);
}
}

```

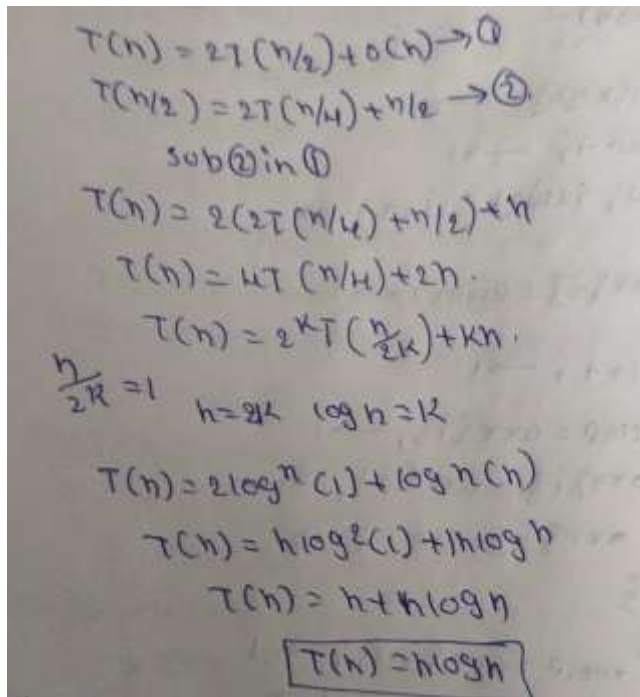
$$\begin{aligned}
 T(n) &= T(n/2) + (n/2) \rightarrow n \\
 T(n) &= 2T(n/2) + n \rightarrow \textcircled{1} \\
 T(n/2) &= 2T(n/4) + n/2 \rightarrow \textcircled{2} \\
 \text{Sub } \textcircled{2} \text{ in } \textcircled{1} \\
 T(n) &= (2T(n/4) + n/2) + n/2 \\
 T(n) &= 2T(n/4) + 2n \rightarrow \textcircled{3} \\
 T(n/4) &= 2T(n/8) + n/2 \rightarrow \textcircled{4} \\
 \text{Sub } \textcircled{4} \text{ in } \textcircled{3} \\
 T(n) &= n(4T(n/16) + n/2) + 2n \\
 &= 16T(n/16) + 2n \\
 &= 16T(n/16) + 4n \\
 T(n) &= 16T(n/16) + 4n \\
 &= 2^4 T(n/2^4) + 4n \\
 &= 2^K T(n/2^K) + Kn \\
 n/2^K &= 1 \\
 n &= 2^K \\
 \text{Apply on both sides} \\
 \log n &= \log 2^K \\
 \log n &= K \log 2 \\
 \log n &= K \\
 T(n) &= 2 \log^n(1) + \log n(n) \\
 T(n) &= n \log 2 + n \log n \\
 &= n(1) + n \log n \\
 \boxed{T(n) &= n \log n}
 \end{aligned}$$

QUICK SORT:

Best case:



Average Case:



$$T(n) = 2T(n/2) + O(n) \rightarrow (1)$$

$$T(n/2) = 2T(n/4) + n/2 \rightarrow (2)$$
 sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1 \quad n = 2^k \quad \log n = k$$

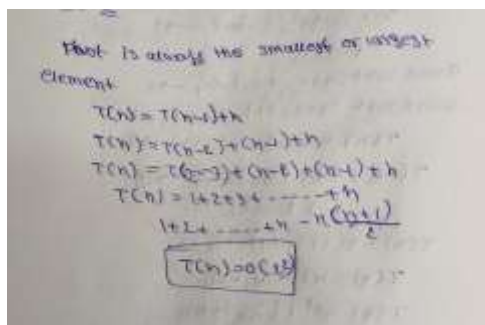
$$T(n) = 2 \log n (1) + \log n (n)$$

$$T(n) = n \log 2 (1) + n \log n$$

$$T(n) = n + n \log n$$

$$T(n) = n \log n$$

Worst Case:



Pivot is always the smallest or largest element

$$T(n) = T(n-1) + n$$

$$T(n) = T(n-2) + (n-1) + n$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n$$

$$T(n) = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

I understood that different sorting algorithms use different ways to arrange data. Bubble, Selection and Insertion Sort are easy to understand and implement, but they can take more time for large data. Merge Sort and Quick Sort use a divide-and-conquer approach and are generally more efficient for large datasets. Overall, choosing an algorithm depends on the size and condition of the data and the require.